INTRODUCTION TO PYTHON
UNIT 5: DATA ANALYSIS WITH PANDAS & VISUALIZATION WITH MATPLOTLIB
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja & McKinney

>> 5.1 The Pandas Data Architecture: Series and DataFrame

Pandas is the primary data manipulation and analytics library for tabular engineering datasets:
* 1. Series: 1-dimensional labeled homogeneous array capable of holding any data type: s = pd.Series(data, index=labels).
* 2. DataFrame: 2-dimensional size-mutable, tabular data structure with labeled axes (rows and columns). Akin to an SQL table or spreadsheet.
* DataFrame Creation: pd.DataFrame(data_dict), pd.read_csv("file.csv"), pd.read_excel("file.xlsx").
* Data Inspection Functions: df.head(n), df.tail(n), df.shape, df.columns, df.info() (memory and null counts), df.describe() (statistical summary: count, mean, std, min, quartiles, max).

>> 5.2 Data Indexing, Slicing and Filtering

* Column Selection: df["col_name"] (returns Series) or df[["col1", "col2"]] (returns subset DataFrame).
* Explicit Indexers:
- df.loc[row_label, col_label]: Label-based indexing (inclusive of end bounds).
- df.iloc[row_idx, col_idx]: Pure integer position-based indexing (exclusive of stop bound).
* Conditional Boolean Filtering: df[(df["voltage"] > 10.0) & (df["temp"] < 50.0)].

>> 5.3 Data Cleaning and Groupby Aggregations

* Handling Missing Data: df.isna().sum(), df.dropna(axis=0), df.fillna(value or df.mean()).
* Data Transformations: df.drop_duplicates(), df.rename(columns={...}), df["col"].apply(custom_func).
* Groupby Operations (Split-Apply-Combine): df.groupby("device_type")["power"].agg(["mean", "max", "count"]).

>> 5.4 Data Visualization Architecture with Matplotlib

* Module: matplotlib.pyplot provides procedural plotting interface.
* Object-Oriented Hierarchy: Figure (overall canvas window) contains one or more Axes (individual subplots with coordinate systems).
* Plot Primitives:
- plt.plot(x, y, label, color, linestyle, marker): Line charts (continuous physical signals).
- plt.scatter(x, y): Discrete data points, correlation analysis.
- plt.bar(categories, values): Categorical comparisons.
- plt.hist(data, bins): Frequency probability distributions.
* Styling & Annotations: plt.title(), plt.xlabel(), plt.ylabel(), plt.grid(True), plt.legend(), plt.xlim(), plt.ylim(), plt.tight_layout(), plt.savefig("plot.png", dpi=300).

[PROGRAM 1: Telemetry Dataset Analysis & Missing Value Cleaning]

Problem Context: Load a noisy experimental sensor DataFrame, clean missing values, compute statistical aggregations, and generate rolling moving averages.
Python Source Code:

```
import pandas as pd
import numpy as np

# Create synthetic laboratory telemetry dataset
data = {
'timestamp_s': [0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
'temperature_c': [24.5, 25.1, np.nan, 26.8, 27.2, 28.0, np.nan, 29.5, 30.1, 31.0],
'battery_v': [12.6, 12.5, 12.4, 12.3, 12.2, 12.1, 12.0, 11.9, 11.8, 11.7],
'current_ma': [120, 125, 130, 250, 128, 132, 135, 300, 138, 140]
}

df = pd.DataFrame(data)
print('Initial DataFrame with Missing Values:\n', df)

# 1. Fill missing temperature values using linear interpolation
df['temperature_c'] = df['temperature_c'].interpolate(method='linear')

# 2. Calculate Power Consumption (P = V * I)
df['power_mw'] = df['battery_v'] * df['current_ma']

# 3. Rolling Moving Average over 3 samples
df['power_smooth_mw'] = df['power_mw'].rolling(window=3, min_periods=1).mean()

print('\nCleaned & Enriched DataFrame:\n', df[[ 'timestamp_s', 'temperature_c',
'power_mw', 'power_smooth_mw' ]])
print('\nStatistical Metrics:\n', df.describe())
```

[PROGRAM 2: Transistor BJT Output Characteristic Curves Plotter]

Problem Context: Model and plot Common Emitter BJT output characteristic curves (IC vs VCE) across varying base currents IB using Matplotlib.

Python Source Code:

```
import matplotlib.pyplot as plt
import numpy as np

# Collector-Emitter Voltage sweep from 0 to 12 Volts
v_ce = np.linspace(0, 12, 500)

# Varying Base Currents in microAmperes
i_b_list = [10, 20, 30, 40, 50] # uA
beta = 100 # Common Emitter Current Gain
v_sat = 0.3 # Saturation voltage

plt.figure(figsize=(9, 5))

for i_b in i_b_list:
i_c_active = (beta * i_b * 1e-3) # mA
# Saturation model: exponential rise to active region
i_c = i_c_active * (1 - np.exp(-v_ce / v_sat)) + 0.05 * v_ce
plt.plot(v_ce, i_c, label=f'IB = {i_b} uA')

plt.title('BJT Common Emitter Output Characteristics (IC vs VCE)', fontsize=12,
fontweight='bold')
plt.xlabel('Collector-Emitter Voltage V_CE (Volts)', fontsize=10)
plt.ylabel('Collector Current I_C (mA)', fontsize=10)
plt.axvline(x=0.3, color='red', linestyle='--', alpha=0.7, label='Saturation Boundary')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper left')
plt.tight_layout()
# plt.savefig('bjt_characteristics.png', dpi=300)
print('Plot configured and styled successfully.')
```

[PROGRAM 3: Multi-Subplot Electrical Waveform Visualization]

Problem Context: Create a professional two-row engineering subplot dashboard displaying AC voltage and current in upper panel and instantaneous power in lower panel.

Python Source Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np

t = np.linspace(0, 0.04, 1000) # 40 ms (2 full 50 Hz cycles)
v = 325.27 * np.sin(2 * np.pi * 50 * t) # 230 V RMS
i = 14.14 * np.sin(2 * np.pi * 50 * t - np.pi / 4) # 10 A RMS lagging 45 deg
p = v * i # Instantaneous power

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)

# Upper Plot: Voltage and Current
ax1.plot(t * 1000, v, 'b-', label='Voltage v(t) [V]')
ax1.set_ylabel('Voltage (V)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

ax1_twin = ax1.twinx()
ax1_twin.plot(t * 1000, i, 'g--', label='Current i(t) [A]')
ax1_twin.set_ylabel('Current (A)', color='green')
ax1_twin.tick_params(axis='y', labelcolor='green')
ax1.set_title('AC Voltage and Current Waveforms', fontweight='bold')
ax1.grid(True, alpha=0.4)

# Lower Plot: Instantaneous Power
ax2.plot(t * 1000, p / 1000, 'r-', label='Power p(t) [kW]')
ax2.axhline(y=np.mean(p) / 1000, color='black', linestyle='--', label=f'Average Power:
{np.mean(p)/1000:.2f} kW')
ax2.set_title('Instantaneous Power Dissipation', fontweight='bold')
ax2.set_xlabel('Time (milliseconds)')
ax2.set_ylabel('Power (kW)')
ax2.grid(True, alpha=0.4)
ax2.legend()

plt.tight_layout()
print('Dual subplot dashboard generated successfully.')
```

[PROGRAM 4: Resistor Quality Control Tolerance Distribution Histogram]

Problem Context: Analyze manufacturing variability of 10 kOhm precision resistors by plotting frequency distribution histogram and calculating yield within +/- 2% tolerance.

Python Source Code:

```
import numpy as np
import pandas as pd

# Simulate batch of 10,000 manufacturing samples (Mean=10000 Ohm, Std=120 Ohm)
np.random.seed(101)
```

```
batch_samples = np.random.normal(loc=10000, scale=120, size=10000)

df_batch = pd.DataFrame(batch_samples, columns=['Resistance_Ohms'])

# Quality Control Limits (+/- 2% of nominal 10,000 Ohm -> [9800, 10200])
in_tolerance = df_batch[(df_batch['Resistance_Ohms'] >= 9800) &
(df_batch['Resistance_Ohms'] <= 10200)]
yield_percentage = (len(in_tolerance) / len(df_batch)) * 100.0

print(f'Total Resistor Batch Size : {len(df_batch)}')
print(f'Batch Mean Resistance : {df_batch['Resistance_Ohms'].mean():.2f} Ohms')
print(f'Batch Standard Deviation : {df_batch['Resistance_Ohms'].std():.2f} Ohms')
print(f'Manufacturing Yield (+/-2%): {yield_percentage:.2f} %')
```